ROADMAP DE ENTREGA

Plano V0 em versões do Hub

Versões rastreáveis do Biofeedback Hub, da demonstração ao pós-MVP

Planejamento atual da HUB-Refinement e fechamento técnico da V0.4

ESTADO	A V0.4 está em fechamento técnico no PR #17. A V0.5 só começa depois da aprovação e do merge da V0.4.

Este material organiza o escopo em incrementos pequenos, com critérios de aceite, branch e evidências para cada etapa.

Status

Documento de planejamento para aprovação antes de iniciar novas implementações em branches separadas.

Este documento organiza o V0 em blocos menores para facilitar rastreamento, validação pelo orientador e abertura de PRs independentes por feature.

Versão anexável: `docs/Plano_V0_Versoes_Hub_atualizado.docx`.

Objetivo

Transformar o escopo do Hub em entregas incrementais, separando:

- o que precisa entrar agora por causa da demonstração;
- o que pertence ao MVP/V0;
- o que deve ficar para V1/V2;
- qual feature deve virar branch e PR separado.

Decisão de fluxo com GitHub

Depois da aprovação deste plano:

- criar uma branch por feature;
- implementar uma feature por vez;
- abrir PR pequeno para cada feature;
- esperar validação do orientador/equipe;
- fazer merge somente depois da validação;
- passar para a próxima feature apenas depois do merge ou decisão explícita da equipe.

Exemplo de fluxo:

```
feature/v0-1-unreal-telemetry-charts
  -> PR
  -> revisão do orientador
  -> merge

feature/v0-2-lsl-architecture-alignment
  -> PR
  -> revisão do orientador
  -> merge
```

Prioridade especial: demonstração

A demonstração em uma semana muda a ordem de execução do V0.

Mesmo que a arquitetura de sensores e LSL seja importante, a prioridade imediata deve ser melhorar a visualização do Hub/dashboard para mostrar melhor os dados da sessão.

Por isso, os gráficos de telemetria da Unreal entram no V0.1, antes das features maiores de Sensor Manager.

V0.1 - Gráficos e visualização da sessão para demonstração

Categoria: V0 imediato / demonstração.

Objetivo:

- melhorar a parte visual do Hub/dashboard para a apresentação;
- mostrar que o Hub consegue observar a qualidade técnica da experiência, não apenas biofeedback;
- facilitar a leitura da sessão pela equipe.

Entram nesta versão:

- gráfico de FPS da Unreal por tempo de experimento;
- gráfico de temperatura do óculos por tempo de experimento;
- gráfico de latência de comunicação;
- gráfico de perda de probes;
- linha de estado da Unreal;
- resumo técnico com FPS, temperatura máxima, latência média, perda máxima e estado mais recente;
- exibição desses dados na área de relatório/dashboard da sessão.

Dados esperados:

- tópico `unreal.state`;
- campos dentro de `payload.telemetry`;
- campos principais: `fps`, `temperatureC`, `latencyMs`, `packetLossPct`;
- estado vindo de `payload.state`, `payload.status`, `payload.telemetry.state` ou `payload.telemetry.status`.

Decisão sobre latência e perda:

- V0.1 implementa a visualização dos campos quando eles chegam no payload.
- `latencyMs` deve ser tratado como latência operacional de comunicação Unreal-Hub, preferencialmente RTT operacional WebSocket, não como latência fisiológica, latência de sensor, render-to-photon ou sincronização LSL.
- `packetLossPct` deve representar perda de probes/respostas de comunicação no nível da aplicação, não perda de pacote de rede em baixo nível.
- A V0.3 implementa a medição real com `system.ping` / `system.pong`: o plugin envia probes, o Hub responde diretamente, e o plugin calcula o RTT com relógio monotônico local.
- Na telemetria V2 da V0.3, zero só é uma medição quando `availability` declara o campo como disponível. Payloads antigos permanecem identificados como legado não verificado.
- Para pesquisa, essas métricas entram como metadados técnicos/contextuais da qualidade da sessão. Elas ajudam a interpretar trechos com instabilidade, atraso ou perda de rastreabilidade, mas não substituem sinais biométricos e não devem ser tratadas como variável fisiológica principal.

Não entra nesta versão:

- correlação automática entre Unreal e biometria;

- alertas automáticos com threshold configurável;
- dashboard científico avançado;
- export estatístico final;
- implementação da medição real de RTT e perda de probes, entregue depois na V0.3.

Critério de aceite:

- a equipe consegue abrir uma sessão e visualizar FPS, temperatura, latência, perda de probes e estado da Unreal;
- os gráficos ajudam a decidir se a sessão teve qualidade técnica mínima;
- a visualização pode ser usada na demonstração sem depender da arquitetura completa de sensores.
- a equipe entende que latência/perda exibidas dependem de telemetria real publicada pela Unreal/plugin.
- a equipe entende que latência/perda são metadados técnicos de apoio à análise da sessão, não métricas fisiológicas.

Branch sugerida:

```
feature/v0-1-unreal-telemetry-charts
```

V0.2 - Alinhamento da arquitetura com LSL

Categoria: V0 arquitetura.

Documento da entrega:

- `docs/lsl-architecture-alignment.md`.

Objetivo:

- definir claramente onde o Lab Streaming Layer entra na arquitetura;
- evitar retrabalho com funções que o LSL já resolve;
- manter o Hub focado em sessão, dashboard, Unreal, markers, logs e relatório.

Entram nesta versão:

- documentar o LSL como adapter/fonte preferencial para sensores compatíveis;
- incluir matriz simples `LSL resolve` / `Hub complementa` / `Hub implementa`;
- separar responsabilidades entre LSL e Hub;
- definir que descoberta, transporte, sincronização e XDF/LabRecorder devem aproveitar LSL quando possível;
- definir que o Hub continua responsável por UX, sessão, attach/detach, preview, comandos, logs e relatórios.
- registrar que sensores sem LSL continuam como rota oficial por `direct_adapter`, com qualidade temporal declarada;
- listar módulos/ideias que devem esperar a análise do LSL;
- registrar recomendação objetiva para V0.1, V0.2 e V0.3.

O LSL ajuda em:

- descoberta de streams;
- leitura de amostras;
- sincronização temporal;
- integração com ferramentas científicas;
- gravação bruta via LabRecorder/XDF em etapas futuras.

O Hub continua necessário para:

- operação simples por dashboard;
- escolha de quais sensores entram na sessão;
- separação entre preview e dado oficial;
- integração com Unreal;
- markers e ciclo da experiência;
- report e export local legível.

Não entra nesta versão:

- implementação de runtime LSL;
- implementação completa do adapter LSL;
- dependência `pylsl`;
- novos tópicos `sensor.*` em `docs/protocol.md`;
- gravação XDF final;
- substituição imediata do driver Polar H10.

Critério de aceite:

- a equipe entende que LSL não substitui o Hub;
- a equipe entende que o Hub não deve ficar refém do LSL: fontes `lsl`, `direct_adapter` e `simulator` são aceitas no desenho;
- a equipe entende que LSL deve ser usado para evitar transporte/sincronização duplicados;
- o documento deixa claro o ponto de entrada do LSL no Sensor Manager.
- o documento pode ser revisado em até 10 minutos e deixa claro o que não deve ser reimplementado agora.

Branch sugerida:

```
feature/v0-2-lsl-architecture-alignment
```

V0.3 - Sensor Manager mínimo

Categoria: V0 base de sensores.

Status: concluída e integrada à `HUB-Refinement` pelo PR #16.

Objetivo:

- criar a base para listar, conectar e controlar sensores pelo Hub/dashboard;
- preparar o projeto para adapters específicos e LSL sem prometer driver universal;

- garantir que lifecycle, sensores e telemetria Unreal permaneçam rastreáveis durante uma sessão.

Entram nesta versão:

- módulo Sensor Manager dentro do processo do Hub;
- adapter `simulator` para validação do fluxo;
- configuração local de sensores/adapters;
- modelo de fonte do sensor: `lsl`, `direct_adapter` ou `simulator`;
- registro de qualidade temporal por fonte: timestamp do sensor, do driver/adapter ou fallback de chegada no Hub;
- estados de sensor: `configured`, `available`, `connecting`, `connected`, `recording`, `stale`, `error`, `disconnected`;
- comandos `sensor.scan`, `sensor.connect`, `sensor.disconnect`, `sensor.session.arm`, `sensor.session.disarm`, `sensor.session.attach` e `sensor.session.detach`;
- publicação de `sensor.status`, `sensor.preview` e `sensor.session`;
- separação entre conectado, preview e recording;
- armamento do sensor antes da experiência e attach automático no primeiro lifecycle `started` válido;
- detach automático no lifecycle `ended` e descarte de amostras atrasadas;
- estado autoritativo em `/status.experienceLifecycle.activeRunId`;
- erros correlacionados `lifecycle_conflict` e `lifecycle_already_ended`;
- recuperação confirmada de run órfão por End pending run, sem apagar logs e sem iniciar outro run automaticamente;
- telemetria Unreal V2 com contexto Editor/Quest e disponibilidade por métrica;
- FPS por janela de um segundo;
- RTT WebSocket real por `system.ping` / `system.pong`;
- perda de respostas sobre probes resolvidos;
- gráficos com `Application FPS`, `Battery temperature`, `WebSocket RTT`, `Probe loss` e linha de estado da Unreal;
- compatibilidade com payloads Unreal e Blueprints antigos.

Não entra nesta versão:

- runtime ou adapter LSL;
- cadastro manual completo pelo dashboard;
- suporte universal a qualquer sensor;
- plugins externos de adapters;
- processamento científico avançado;
- métricas automáticas de CPU/GPU;
- gráfico de memória ou bateria percentual.

Critério de aceite:

- o dashboard consegue mostrar sensores configurados/disponíveis, preview e estado de gravação;

- o usuário entende se o sensor está apenas conectado, armado ou gravando oficialmente;
- preview não entra no JSONL como dado oficial;
- o Hub rejeita runs concorrentes antes do log e do broadcast;
- um run órfão pode ser encerrado com confirmação, detach dos sensores e preservação do histórico;
- uma nova experiência só começa após outro clique do operador;
- Editor não apresenta bateria/temperatura indisponível como zero;
- Quest apresenta contexto Android/Packaged e métricas disponíveis quando fornecidas pela plataforma;
- `system.ping` e `system.pong` permanecem efêmeros e fora do contador de ACK;
- RTT e perda de probes são metadados técnicos, não latência fisiológica ou sincronização LSL.

Branch sugerida:

```
feature/v0-3-sensor-manager-minimal
```

V0.4 - Adapter LSL mínimo

Categoria: V0 integração LSL.

Status: em fechamento técnico na branch `feature/v0-4-lsl-adapter-minimal`; o PR #17 permanece aberto para revisão.

Objetivo:

- permitir que streams LSL apareçam como fontes/sensores no Hub.

Entram nesta versão:

- `lsl_adapter` mínimo;
- provider opcional, que não impede o Hub de iniciar sem `pylsl`;
- descoberta dinâmica de todos os streams LSL numéricos;
- um sensor gerenciado por stream suportado;
- leitura de metadados, canais, labels e unidades quando disponíveis;
- preview de dados sem persistência oficial;
- attach/detach para gravar lotes `lsl.raw` apenas quando o stream entra na sessão;
- preservação do timestamp LSL original, correção temporal e horário de chegada;
- outlet numérico local para teste reproduzível.
- fila limitada do logger com erro terminal e flush no encerramento;
- reconexão da telemetria e do lifecycle da ponte Polar, sem repetição automática de comandos de gravação;
- recuperação de consumidor do Sensor Manager e encerramento protegido contra cancelamento;
- separação do histórico operacional e do histórico associado a cada `runId` no dashboard.
- correlação de ACK tardio com o comando e o `runId` de origem, sem alterar relatórios de execuções já encerradas;

Não entra nesta versão:

- pipeline completo XDF;
- configuração visual avançada de canais;
- reprocessamento dos dados LSL;
- suporte a streams de texto/markers;
- mapeamento automático para ECG, EEG ou HRV;
- gráficos específicos por canal;
- suporte científico completo a streams de alta frequência.

Critério de aceite:

- todos os streams numéricos de teste aparecem como cards independentes;
- o Hub continua operacional quando o suporte LSL está ausente;
- o stream conectado mostra metadados e preview;
- `sensor.preview` continua fora do JSONL;
- lotes `lsl.raw` entram somente depois de attach e param após detach;
- timestamps original e corrigido permanecem rastreáveis.
- falhas de disco não alteram o lifecycle autoritativo nem distribuem transições não persistidas;
- falha no endpoint de controle Polar não encerra a telemetria;
- o histórico e o relatório não misturam comandos de runs diferentes.
- um ACK tardio pode reconciliar o item original após timeout, enquanto Clear history afeta somente a lista operacional.

Branch sugerida:

```
feature/v0-4-lsl-adapter-minimal
```

V0.5 - Polar H10 e sensores reais já validados

Categoria: V0 continuidade dos testes reais.

Objetivo:

- manter funcionando o que já foi validado com sensores reais;
- reorganizar sem quebrar o fluxo atual da Polar H10.

Entram nesta versão:

- manter integração com Polar H10 pelo driver atual ou por adapter específico;
- preservar separação entre HRV e ECG bruto;
- manter timestamps corrigidos sem reintroduzir esse problema;
- registrar status, qualidade, gaps e contagem de amostras;
- garantir que sensor só grave oficialmente quando entrar na sessão.

Não entra nesta versão:

- refazer todo o driver Polar H10 sem necessidade;

- mexer em timestamp já resolvido;
- exigir LSL para tudo se isso atrasar a demo ou quebrar sensor validado.

Critério de aceite:

- uma sessão real com Polar H10 continua coletando;
- HRV e ECG bruto continuam separados;
- o report identifica sensor, tópicos e amostras da sessão.

Branch sugerida:

```
feature/v0-5-polarh10-sensor-session
```

V0.6 - Report e export mínimos

Categoria: V0 rastreabilidade.

Objetivo:

- garantir que a equipe consiga encontrar e interpretar os dados coletados.

Entram nesta versão:

- seção "Sensores da sessão" no report;
- seção de telemetria Unreal no report;
- indicação de `runId`, `sessionId`, tópicos e arquivos relacionados;
- export JSON/JSONL suficiente para conferência;
- resumo de qualidade, gaps e avisos importantes.

Não entra nesta versão:

- formato científico final versionado;
- pacote completo com manifest científico;
- visualizador local avançado de JSONL.

Critério de aceite:

- a equipe consegue decidir se a sessão é analisável;
- a equipe consegue localizar dados de Unreal, sensores, markers e lifecycle.

Branch sugerida:

```
feature/v0-6-session-report-export
```

V0.7 - Preflight visual

Categoria: V0 operação.

Objetivo:

- reduzir erro operacional antes do teste.

Entram nesta versão:

- aviso de Hub online/offline;
- aviso de Unreal conectada/desconectada;
- aviso de sensor sem preview recente;
- aviso de sensor conectado mas fora da sessão;
- aviso de stream LSL disponível mas não selecionado;
- confirmação visual do que será gravado antes de iniciar.

Não entra nesta versão:

- bloqueio rígido de início da sessão;
- regras avançadas por perfil de experimento;
- thresholds configuráveis completos.

Critério de aceite:

- antes de iniciar, o usuário vê claramente o que será gravado e o que ficará fora.

Branch sugerida:

```
feature/v0-7-preflight-visual
```

V0.8 - Documentação operacional da demonstração

Categoria: V0 documentação.

Objetivo:

- permitir que outra pessoa da equipe rode a demo com menor dependência técnica.

Entram nesta versão:

- passo a passo para iniciar Hub;
- passo a passo para iniciar dashboard;
- passo a passo para conectar Unreal;
- passo a passo para sensores/streams usados na demonstração;
- como validar gráficos Unreal;
- como anexar report/logs da sessão;
- explicação curta de como LSL se encaixa no plano.

Não entra nesta versão:

- manual completo de todos os adapters;
- documentação final da API pública;
- guia completo de contribuição open source.

Critério de aceite:

- uma pessoa da equipe consegue reproduzir a demonstração seguindo o documento.

Branch sugerida:

```
feature/v0-8-demo-operational-docs
```

V1 / pós-MVP

Ficam para depois do V0:

- cadastro manual de sensores pelo dashboard;
- wizard "Adicionar sensor";
- adapters genéricos mais completos: WebSocket, Serial/USB, BLE e UDP;
- integração completa com LabRecorder/XDF;
- perfis locais de experimento;
- presets por tipo de sessão;
- tela de calibração por sensor;
- Tratador de Dados inicial;
- alertas configuráveis de qualidade, stale, bateria, perda de amostras e sample rate;
- export científico mais estruturado.

V2 / V3

Ficam como visão futura:

- plugins externos de adapters;
- catálogo de adapters da comunidade;
- instalação de adapters externos;
- perfis por usuário, projeto ou laboratório;
- pipeline científico versionado;
- reprocessamento avançado de dados brutos;
- sincronização com servidor/plataforma;
- multi-participante e multi-sessão avançado;
- permissões, papéis e auditoria;
- templates compartilháveis de experimento.

Ordem de execução atual

Entregas concluídas e integradas à `HUB-Refinement`:

1. V0.1 - Gráficos e visualização da sessão.
2. V0.2 - Alinhamento da arquitetura com LSL.
3. V0.3 - Sensor Manager mínimo, telemetria Unreal V2 e recuperação de lifecycle.

Entrega em fechamento técnico:

4. V0.4 - Adapter LSL mínimo, hardening do runtime, ponte Polar e isolamento do histórico no dashboard, no PR #17.

Próximas entregas, somente depois da aprovação e do merge da V0.4:

5. V0.5 - Polar H10 e sensores reais já validados.

6. V0.6 - Report e export mínimos.
7. V0.7 - Preflight visual.
8. V0.8 - Documentação operacional consolidada.

Critério de aceite do plano

Este plano está pronto para orientar a implementação quando:

- o orientador aprovar a divisão em V0.1, V0.2, V0.3 etc.;
- a equipe concordar que gráficos Unreal entram primeiro por causa da demonstração;
- a equipe concordar que LSL entra como adapter/fonte, não como substituto total do Hub;
- a equipe concordar com uma branch e um PR por feature;
- a equipe revisar e aprovar cada PR antes do merge;
- a V0.5 só começar depois do fechamento aprovado e do merge da V0.4.